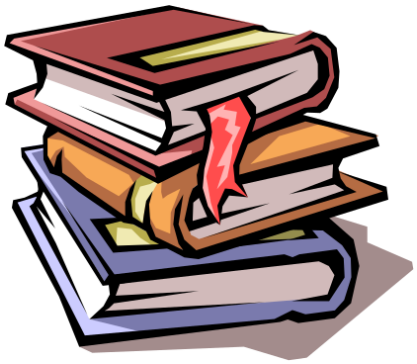
FPT UNIVERSITY

Lab Report

LAB-01 – CẤU HÌNH KERNEL & KHỞI ĐỘNG HỆ THỐNG NHÚNG

Embedded Linux Systems – ARMv7 Cortex-A9 (QEMU vexpress-a9)



Student Name	Phạm Võ Hoàng Nhân
Student ID	SE200778
Class	IC2008
Lecturer	Phạm Thế Vinh
Date	27/09/2026

1. Thông tin sinh viên và môi trường

Họ và tên	Phạm Võ Hoàng Nhân
MSSV / Lớp	SE200778 / IC2008
Ngày thực hiện	27/09/2026
Máy host	Ubuntu 26.04, GCC 15.2, toolchain `arm-linux-gnueabihf`
Target	QEMU `vexpress-a9`, CPU Cortex-A9 (ARMv7-A)
Mã nguồn	Linux 5.15, U-Boot 2022.04, BusyBox 1.35.0

Đề bài viết cho Ubuntu 20.04/22.04. Máy em chạy Ubuntu 26.04 với GCC 15.2, mới hơn kernel 5.15 và BusyBox 1.35 khá nhiều, nên em phải sửa hai chỗ trong hệ build mới chạy được. Hai lỗi này được trình bày ở mục 5.

2. Sơ đồ chuỗi khởi động

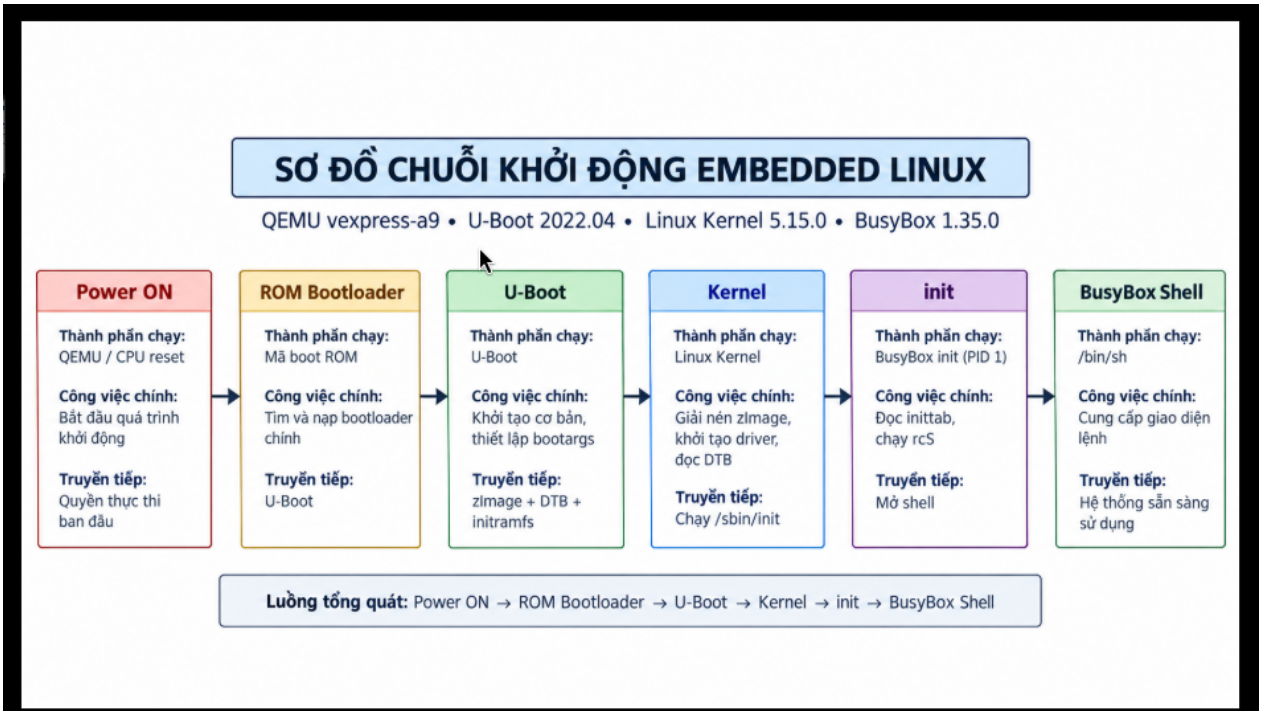


Figure 1: Chuỗi khởi động từ lúc cấp nguồn đến BusyBox shell

Khi QEMU bật máy, CPU Cortex-A9 được reset và nhảy vào mã khởi động đầu tiên. Trên board thật đây là boot ROM; trên QEMU, phần này được giả lập và chuyển ngay sang U-Boot, file mà em truyền vào bằng `-kernel u-boot`. U-Boot khởi tạo phần cứng tối thiểu (UART, bộ nhớ), đọc các biến môi trường rồi chạy `bootcmd`. Trong bài này `bootcmd` là

```
bootz 0x60100000 0x64000000 0x60000000
      zImage     initramfs  DTB
```

nghĩa là U-Boot nhảy vào zImage ở `0x60100000`, đồng thời báo cho kernel biết initramfs nằm ở `0x64000000` và Device Tree nằm ở `0x60000000`. Chuỗi `bootargs` (console, root, init, mem) được U-Boot ghi vào DTB để kernel đọc như kernel command line.

Kernel tự giải nén zImage, gọi `start_kernel()`, đọc DTB để biết phần cứng có gì (UART PL011, RAM, timer…), khởi tạo driver, rồi giải nén initramfs vào RAM làm root filesystem. Bước

cuối của kernel là chạy `/sbin/init`, đây là lúc chuyển từ kernel space sang user space. BusyBox init (PID 1) đọc `/etc/inittab`, chạy `/etc/init.d/rcS` để mount `/proc`, `/sys`, `/tmp` và đặt hostname, sau đó mở `/bin/sh` trên console. Từ đây hệ thống sẵn sàng nhận lệnh.

3. Log thực hành

Phần A – Build kernel

Em dùng `vexpress_defconfig` chứ không dùng `versatile_defconfig` như trong danh sách nộp bài, vì target của bài là vexpress-a9 (Cortex-A9). Versatile PB là board ARM926 (ARMv5), không khớp với CPU đang dùng.

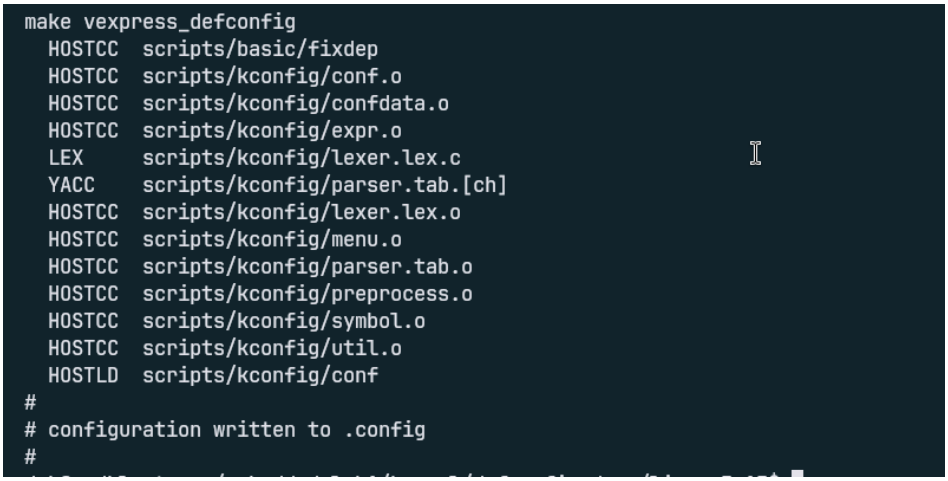


Figure 2: Output của `make vexpress_defconfig`

Trong menuconfig em xác nhận `CONFIG_ARCH_VEXPRESS=y`, bật hỗ trợ initramfs (General setup → Initial RAM filesystem and RAM disk support), giữ driver UART PL011 để có console `ttyAMA0`, và bật SMP.

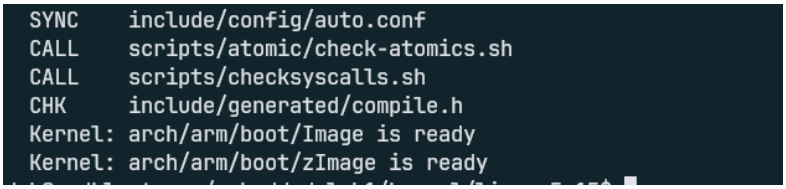


Figure 3: Cuối log build: `Image` và `zImage` đã sẵn sàng



Figure 4: Kích thước zImage sau khi build: 4,9 MB

zImage nặng 4,9 MB, nằm trong khoảng 5–8 MB mà đề nói là bình thường cho vexpress_defconfig. Chuỗi `SMP` trong `uname -a` ở phần C cũng xác nhận đúng cấu hình đã build.

Phần B – U-Boot



Figure 5: `printenv` trên U-Boot (phần đầu)

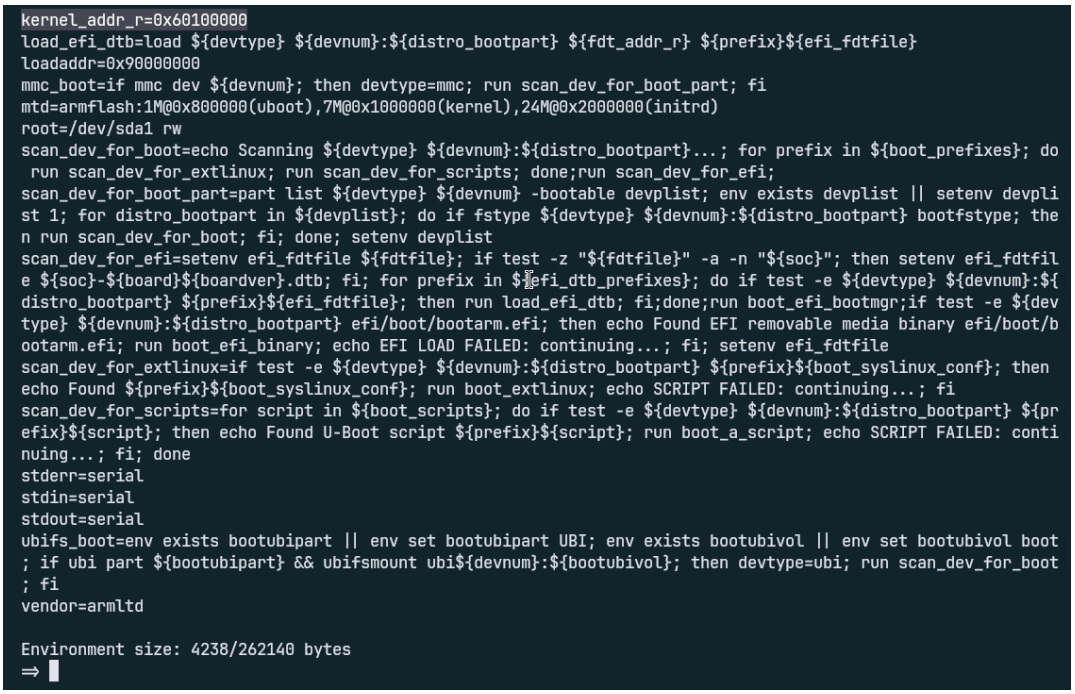


Figure 6: `printenv` (phần cuối)

Môi trường mặc định của U-Boot cho vexpress đã có sẵn các địa chỉ nằm trong vùng RAM `0x6xxxxxxx`: `kernel_addr_r=0x60100000`, `fdt_addr_r=0x60000000`, `fdtfile=vexpress-v2p-ca9.dtb`, `cpu=armv7`. `bootargs` mặc định chỉ có console ở 38400 baud, nên em đặt lại:



Figure 7: `setenv bootargs` và kiểm tra lại bằng `printenv bootargs`

Khi boot thử lúc chưa có root filesystem, kernel chạy hết phần khởi tạo rồi dừng ở bước mount root:



Figure 8: Kernel panic khi chưa có rootfs

Đây là kết quả đúng như đề yêu cầu quan sát. `bootargs` bảo kernel dùng `root=/dev/ram0`, nhưng không có initramfs nào được nạp vào RAM nên `/dev/ram0` rỗng. Kernel liệt kê các block device nó tìm thấy (`mtdblock0`, `mtdblock1` là flash giả lập của QEMU), không mount được cái nào làm root, và `mount_block_root` gọi `panic()`. Backtrace cho thấy đường đi `kernel_init` → `prepare_namespace` → `mount_block_root` → `panic`, tức là lỗi xảy ra ở đúng bước "mount rootfs" trong sơ đồ ở mục 2. Kernel vẫn đang chạy ở PID 1 (`swapper/0`), chưa bước được sang user space.

Phần C – Root filesystem và boot đến shell

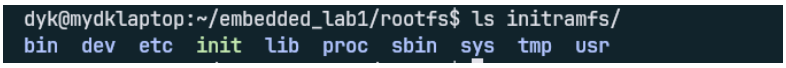


Figure 9: Cấu trúc thư mục `initramfs/`

BusyBox được build static (`CONFIG_STATIC=y`), nên rootfs không cần thư mục thư viện nào có nội dung. Sau khi đóng gói thành `initramfs.cpio.gz`, em boot bằng QEMU với `-kernel zImage -dtb vexpress-v2p-ca9.dtb -initrd initramfs.cpio.gz`.

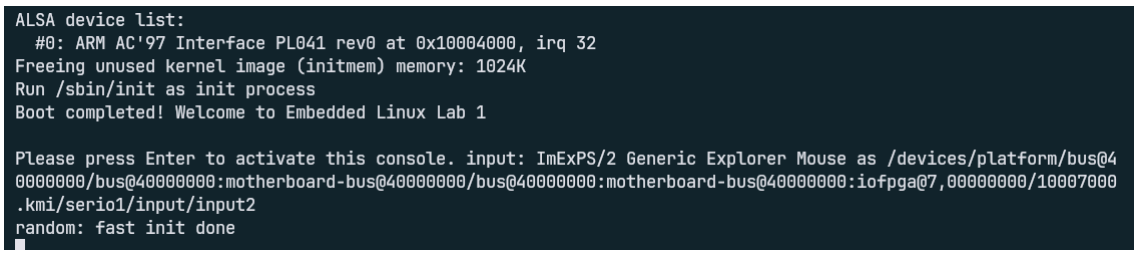


Figure 10: Boot thành công: kernel chạy `/sbin/init`, rcS in thông báo



Figure 11: `uname -a`: kernel 5.15.0, SMP, kiến trúc `armv7l`

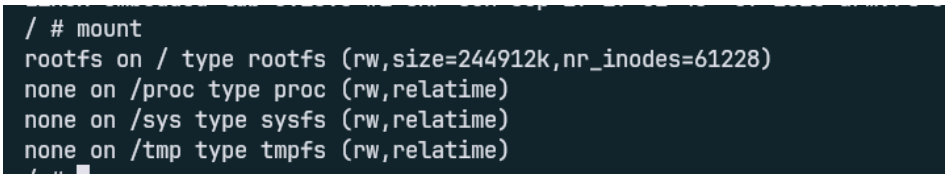


Figure 12: `mount`: rootfs, proc, sysfs, tmpfs

`mount` cho thấy ba filesystem mà rcS mount (`proc`, `sysfs`, `tmpfs`) cộng với `rootfs` chính là initramfs đã giải nén vào RAM.

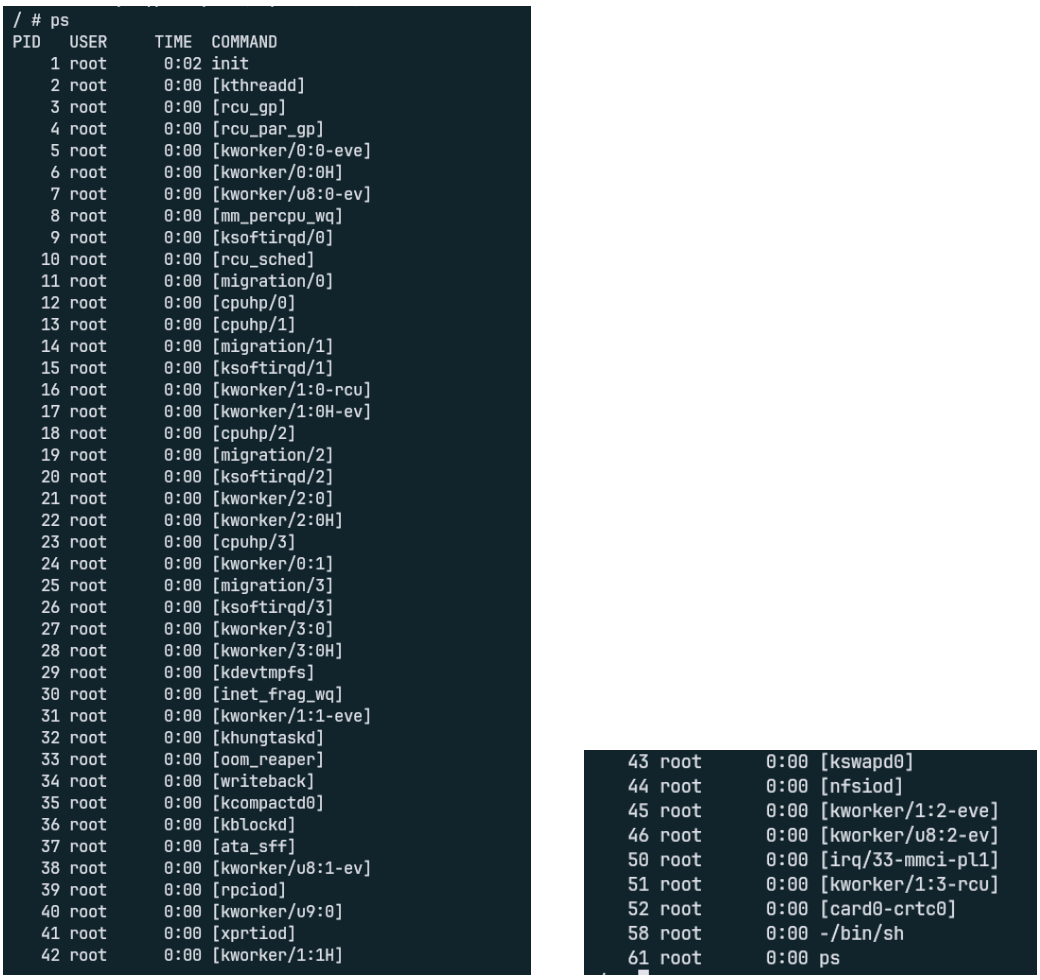


Figure 13: `ps`: PID 1 là `init`, phần lớn còn lại là kernel thread

Trong `ps` chỉ có ba tiến trình user space: `init` (PID 1), `-/bin/sh` do inittab mở, và chính lệnh `ps`. Các dòng trong ngoặc vuông là kernel thread. Đáng chú ý là các thread theo từng CPU như `migration/N`, `ksoftirqd/N`, `cpuhp/N` xuất hiện với N từ 0 đến 3, tức là kernel đang chạy trên 4 CPU (xem câu 5).

4. Trả lời câu hỏi phân tích

Câu 1. arm-linux-gnueabi và arm-linux-gnueabihf

Hai toolchain khác nhau ở ABI dấu phẩy động. `gnueabi` dùng soft-float ABI: tham số và giá trị trả về kiểu `float`/`double` được truyền qua thanh ghi số nguyên (r0–r3), và mặc định nhắm tới kiến trúc cũ (ARMv5TE) không giả định có FPU. `gnueabihf` dùng hard-float ABI: tham số dấu phẩy động đi thẳng qua thanh ghi VFP (s0–s15/d0–d7), và mặc định nhắm tới ARMv7-A có VFPv3. Hai ABI này không trộn được với nhau trong cùng một chương trình, vì hàm gọi và hàm được gọi phải đồng ý tham số nằm ở thanh ghi nào.

Cortex-A9 có VFPv3 và NEON, nên dùng `gnueabihf` thì phép tính float chạy trực tiếp trên FPU và không tốn công chép giá trị qua lại giữa thanh ghi số nguyên và thanh ghi VFP.

Riêng với kernel thì câu chuyện khác một chút: kernel Linux không dùng float, nó luôn được build với `-msoft-float` (em thấy đúng cờ này trong log build ở mục 5). Vì vậy nếu build kernel bằng `gnueabi` với `ARCH=arm` và cấu hình ARMv7, kernel vẫn build được và vẫn boot được. Vấn đề nằm ở user space: BusyBox và các chương trình khác build bằng `gnueabi` sẽ dùng soft-float, chạy

được trên Cortex-A9 nhưng không tận dụng FPU, và nếu build dynamic thì không thể dùng chung thư viện với các chương trình hard-float. Cờ `KCFLAGS` có `-mfloat-abi=hard` mà đề gợi ý cũng vì lý do này mà không hợp với kernel, nên em đã bỏ nó (`unset KCFLAGS`).

Câu 2. Memory map bắt đầu từ 0x60000000

Versatile PB đặt SDRAM ở đầu không gian địa chỉ, nên RAM bắt đầu từ `0x00000000`. Vexpress với daughterboard CoreTile Cortex-A9 (V2P-CA9) dùng bản đồ địa chỉ khác: vùng thấp được dành cho flash NOR để boot và cho các thiết bị ngoại vi của motherboard (ví dụ UART PL011 ở `0x10009000`, âm thanh PL041 ở `0x10004000` như trong log boot), còn DRAM của daughterboard được đặt ở `0x60000000`–`0x9FFFFFFF` (1 GB).

Vì vậy mọi địa chỉ nạp trong U-Boot phải nằm trong cửa sổ này. Em thấy điều đó ngay trong `printenv`: `kernel_addr_r=0x60100000`, `fdt_addr_r=0x60000000`. Lệnh `bootz 0x60100000 0x64000000 0x60000000` đặt DTB ở đầu RAM, zImage cách đó 1 MB, và initramfs ở `0x64000000` (cách zImage khoảng 63 MB, đủ chỗ cho kernel tự giải nén mà không đè lên initramfs). Nếu dùng nguyên các địa chỉ kiểu `0x00008000` của Versatile PB, U-Boot sẽ ghi vào vùng flash/ngoại vi chứ không phải RAM và kernel không chạy được. Kernel cũng cần biết RAM bắt đầu ở đâu; thông tin này nằm trong node `memory` của DTB, và vì kernel 5.15 bật `ARM_PATCH_PHYS_VIRT` (chữ `p2v8` trong vermagic ở LAB-02) nên cùng một zImage chạy được ở địa chỉ vật lý nào cũng được.

Câu 3. MIDR 0x410FC090

Main ID Register là thanh ghi chỉ đọc cho biết lõi CPU là gì. Tách 32 bit của `0x410FC090`:

Bit	Trường	Giá trị	Ý nghĩa
31–24	Implementer	0x41	ARM Ltd. (chữ ‘A’)
23–20	Variant	0x0	r0 (bản lớn)
19–16	Architecture	0xF	khai báo theo CPUID scheme, tức ARMv7
15–4	Part number	0xC09	Cortex-A9
3–0	Revision	0x0	p0 (bản nhỏ)

Vậy QEMU đang giả lập Cortex-A9 bản r0p0. Kernel dùng part number để chọn đúng `proc_info` cho CPU (cache, TLB, các errata cần vá) khi khởi động. Cùng giá trị này cũng hiện trong `/proc/cpuinfo` dưới dạng `CPU implementer: 0x41`, `CPU part: 0xc09`.

Câu 4. Tại sao BusyBox build static

Initramfs tối giản của bài chỉ có BusyBox, không có thư viện C hay dynamic linker. Một binary dynamic khi chạy sẽ gọi trình nạp `/lib/ld-linux-armhf.so.3` để nạp các thư viện `.so`; nếu thiếu file này, kernel báo “not found” ngay khi exec `/sbin/init` và hệ thống không vào

được user space. Build static thì mọi thứ nằm trong một file duy nhất, rootfs gọn và không phụ thuộc gì.

Nếu build dynamic, cần chép từ sysroot của toolchain (`/usr/arm-linux-gnueabihf/lib/`) vào `rootfs/lib/` ít nhất: `ld-linux-armhf.so.3` (dynamic linker), `libc.so.6` (thư viện C), `libm.so.6` (hàm toán học BusyBox dùng), và tùy cấu hình thêm `libresolv.so.2`. Cách chắc nhất để biết đủ chưa là chạy `arm-linux-gnueabihf-readelf -d busybox | grep NEEDED` rồi chép đúng các file được liệt kê.

Câu 5. SMP trên vexpress-a9

Với SMP, kernel phải làm thêm mấy việc mà bản đơn nhân không có. CPU0 boot trước, sau đó kernel đọc số CPU từ DTB, bật SCU (Snoop Control Unit) để giữ cache của các lõi nhất quán, rồi đánh thức từng CPU phụ: ghi địa chỉ khởi động vào thanh ghi system flags của vexpress và gửi ngắt IPI qua GIC. Mỗi CPU phụ chạy `secondary_start_kernel()`, khởi tạo timer local và vùng dữ liệu per-CPU của riêng nó. Scheduler phải tạo runqueue cho từng CPU và các kernel thread theo CPU (`migration`, `ksoftirqd`, `cpuhp`, `kworker`). Các spinlock lúc này mới thực sự khóa, còn trên bản đơn nhân chúng gần như chỉ là tắt preemption.

Bằng chứng trong bài của em là output `ps` ở phần C: có `migration/0..3`, `ksoftirqd/0..3`, `cpuhp/0..3`, tức kernel đã bật 4 CPU. Với `-smp 1` các thread này chỉ có bản `/0`. Trong boot log, bản SMP có thêm các dòng kiểu "CPU1: thread -1, cpu 1, socket 0" cho từng lõi và "SMP: Total of N processors activated".

Về giá phải trả: image lớn hơn một chút vì có thêm mã SMP (IPI, per-CPU, khóa thật), và boot lâu hơn một ít vì phải chờ từng CPU phụ lên. Trên một hệ nhúng chỉ có một lõi, tắt SMP giúp image nhỏ và nhanh hơn.

5. Nhận xét và khó khăn gặp phải

Lỗi 1: kernel 5.15 không build được với GCC 15

Lần build đầu, assembler báo hàng loạt lỗi kiểu

```
Error: selected processor does not support `dsb st' in ARM mode
Error: selected processor does not support `dmb ish' in ARM mode
Error: selected processor does not support `ldrex' ...
```

Các lệnh `dsb`, `dmb`, `isb`, `ldrex`, `strex` đều là lệnh của ARMv7, nên lỗi nghĩa là compiler đang nghĩ target là một kiến trúc cũ hơn. Khi xem dòng lệnh compile đầy đủ trong log, em thấy

đúng như vậy: `-D__LINUX_ARM_ARCH__=7 -march=armv5t -Wa,-march=armv7-a -msoft-float`.

Nguyên nhân nằm ở `arch/arm/Makefile` của kernel 5.15:

```
arch-$(CONFIG_CPU_32v7) =-D__LINUX_ARM_ARCH__=7 \
  $(call cc-option,-march=armv7-a,\
         -march=armv5t -Wa$(comma)-march=armv7-a)
```

`cc-option` thử xem compiler có nhận `-march=armv7-a` không, nếu không thì lùi về `-march=armv5t`. Với cross-GCC hard-float đời mới, phép thử này thất bại dù GCC thực ra hỗ trợ ARMv7-A, nên kernel lùi về ARMv5T và assembler từ chối các lệnh ARMv7. Em giữ nguyên kernel 5.15 theo đề, chỉ sửa đúng dòng của ARMv7 thành

```
arch-$(CONFIG_CPU_32v7) =-D__LINUX_ARM_ARCH__=7 -march=armv7-a
```

(bản kernel mainline mới cũng đã đơn giản hóa dòng này tương tự). Các dòng ARMv6, ARMv5 em không đụng vì target không dùng tới. Build lại thì ra `Kernel: arch/arm/boot/zImage is ready`.

Lỗi 2: BusyBox menuconfig báo thiếu ncurses

`make menuconfig` của BusyBox 1.35 báo “Unable to find the ncurses libraries” dù máy đã cài `libncurses-dev`. Script `scripts/kconfig/lxdialog/check-lxdialog.sh` kiểm tra ncurses bằng cách biên dịch một đoạn C có dòng `main() {}`. Từ GCC 14, hàm không khai báo kiểu trả về (implicit int) bị coi là lỗi chứ không còn là cảnh báo, nên đoạn thử biên dịch lỗi và script kết luận sai là thiếu ncurses. Em sửa thành `int main() { return 0; }` bằng `sed`, sau đó menuconfig chạy bình thường.

Nhận xét

Cả hai lỗi đều không phải do cấu hình sai mà do mã nguồn cũ gặp compiler mới, và cách tìm ra đều giống nhau: đọc dòng lệnh compile thật trong log thay vì chỉ đọc thông báo lỗi cuối. Phần U-Boot cho em thấy rõ vai trò của từng thành phần trong chuỗi boot: kernel panic ở phần B không phải lỗi của kernel mà là thiếu một mắt xích (rootfs), và chỉ cần nạp initramfs là hệ thống đi tiếp được đến shell.